



Material de Estudio — Parcial 2

Guía de estudio para comprender el funcionamiento de la plataforma (SvelteKit) y los conceptos del curso.

Guía de Estudio: Desarrollo Web 1 con SvelteKit

Plataforma Educativa Cinar Sistemas — Parciales

Duración de la clase: 2 horas (≈20 min por tema)

Proyecto de referencia: `testcinar26` — Plataforma educativa con autenticación JWT, dashboard, exámenes parciales, algoritmia y enlaces de consulta.

Tema 1 — Navegación en la Aplicación (20 min)

Implementación en el proyecto

Archivo	Líneas	Rol
<code>src/routes/+layout.svelte</code>	1-122	Layout global
<code>src/routes/Header.svelte</code>	1-176	Barra de navegación
<code>src/routes/+page.svelte</code>	1-699	Página home
<code>src/routes/login/+page.svelte</code>	1-278	Login
<code>src/routes/dashboard/+page.svelte</code>	1-968	Dashboard admin
<code>src/routes/desarrollo-web-1/+page.svelte</code>	1-405	Menú módulos
<code>src/routes/desarrollo-web-1/parcial-1/+page.svelte</code>	1-200	Examen parcial
<code>src/routes/desarrollo-web-1/algoritmia/+page.svelte</code>	1-120	Algoritmia

Comandos Svelte

```
# Crear proyecto SvelteKit
npx sv create mi-app

# Crear ruta
mkdir -p src/routes/mi-ruta
# Luego crear src/routes/mi-ruta/+page.svelte

# Layout ya existe en src/routes/+layout.svelte
# Componentes reutilizables en src/lib/components/
```

Teoría: Sistema de enrutamiento en SvelteKit

SvelteKit utiliza **filesystem-based routing**: la estructura de carpetas define las rutas URL.

Archivos especiales con prefijo `+`:

- `+page.svelte` — Define una página (ruta navegable)
- `+layout.svelte` — Layout que envuelve páginas hijas
- `+error.svelte` — Página de error para la ruta
- `+page.ts` / `+page.server.ts` — Load function (carga datos antes de renderizar)
- `+layout.ts` / `+layout.server.ts` — Load function del layout
- `+server.ts` — Endpoint API (GET, POST, PUT, DELETE)

Rutas dinámicas (parametrizadas):

```
src/routes/usuario/[id]/+page.svelte → /usuario/123
src/routes/usuario/[id]/[post]/+page.svelte → /usuario/123/mi-post
```

Navegación entre rutas:

1. `<a href="/ruta">` — Etiqueta HTML estándar (SvelteKit intercepta la navegación)
2. `goto('/ruta')` — Navegación programática desde `$app/navigation`
3. `redirect(303, '/ruta')` — Redirección desde load functions (server-side)

Layouts anidados: Se pueden tener layouts en diferentes niveles. Cada layout envuelve las páginas de su subárbol.

1.1 Enrutamiento basado en archivos

```
src/routes/
+page.svelte      → /
+layout.svelte    → Layout global
login/+page.svelte → /login
dashboard/+page.svelte → /dashboard
desarrollo-web-1/
  +page.svelte      → /desarrollo-web-1
  parcial-1/+page.svelte → /desarrollo-web-1/parcial-1
  algoritmia/+page.svelte → /desarrollo-web-1/algoritmia
```

1.2 Layout (`+layout.svelte`) — Líneas 1-122

```
<script lang="ts">
  import { onMount } from 'svelte';
  import Header from './Header.svelte';
  let { children } = $props(); // slots en Svelte 5
</script>

<div class="app">
  <Header />
  <main>
    {@render children()} {/* Renderiza contenido de cada página */}
  </main>
</div>
```

1.3 Navegación con `<a>` — Header.svelte:13-41

```
<a href="/">Inicio</a>
<a href="/dashboard">Calificaciones</a>
<a href="/desarrollo-web-1">Desarrollo Web 1</a>
```

1.4 Navegación con `goto()` — `Header.svelte:9`, `+page.svelte:5`

```
<script lang="ts">
  import { goto } from '$app/navigation';
  function handleLogout() { logout(); goto('/login'); }
</script>
<button onclick={() => goto('/')}>Volver</button>
```

Tema 2 — Configuración de la Página y SEO (20 min)

📌 Implementación

Archivo	Líneas	Rol
<code>src/app.html</code>	1-13	Plantilla HTML base
<code>src/routes/+layout.ts</code>	1-15	Load function layout
<code>src/routes/+page.ts</code>	1-24	Load function home
<code>src/routes/+page.svelte</code>	102-104	<code><svelte:head></code>
<code>src/routes/login/+page.svelte</code>	29-31	<code><svelte:head></code>
<code>src/routes/dashboard/+page.svelte</code>	230-232	<code><svelte:head></code>
<code>src/routes/desarrollo-web-1/+page.svelte</code>	19-21	<code><svelte:head></code>

📖 Teoría: SEO y metadatos en SvelteKit

`app.html` es la plantilla HTML raíz. Contiene placeholders:

- `%sveltekit.head%` — Donde SvelteKit inyecta etiquetas `<title>`, `<meta>`, `<link>`
- `%sveltekit.body%` — Donde se renderiza la aplicación
- `%sveltekit.assets%` — Ruta a archivos estáticos

`<svelte:head>` permite inyectar etiquetas en el `<head>` desde cualquier componente:

- `<title>` — Título de la página (crítico para SEO)
- `<meta name="description">` — Descripción para buscadores
- `<meta property="og:...">` — Open Graph para redes sociales
- `<link rel="canonical">` — URL canónica
- `<script type="application/ld+json">` — Datos estructurados

Load functions (`+page.ts`, `+layout.ts`):

- Se ejecutan antes de renderizar la página
- Pueden ser CSR (cliente) o SSR (servidor con `+page.server.ts`)
- Devuelven datos que se reciben como `let { data } = $props()`
- Útiles para precargar datos y mejorar SEO (SSR)

2.1 app.html — Líneas 1-13

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  %sveltekit.head%
</head>
<body>
  <div style="display: contents"%sveltekit.body%</div>
</body>
</html>
```

2.2 <svelte:head> en cada página

```
<!-- +page.svelte:102-104 -->
<svelte:head>
  <title>Inicio - Cinar Sistemas</title>
</svelte:head>

<!-- login/+page.svelte:29-31 -->
<svelte:head>
  <title>Iniciar Sesión</title>
</svelte:head>

<!-- dashboard/+page.svelte:230-232 -->
<svelte:head>
  <title>Calificaciones - Cinar Sistemas</title>
</svelte:head>

<!-- desarrollo-web-1/+page.svelte:19-21 -->
<svelte:head>
  <title>Desarrollo Web 1 — Cinar Sistemas</title>
</svelte:head>
```

2.3 Load functions — +layout.ts:1-15, +page.ts:1-24

```
import { browser } from '$app/environment';
import type { PageLoad } from './$types';
import { gradesApi } from '$lib/api';

export const load: PageLoad = async () => {
  if (browser) {
    const grades = await gradesApi.getMine();
    return { grades };
  }
  return { grades: [] };
};
```

Tema 3 — Manejo de Datos (20 min)

Implementación

Archivo	Líneas	RoI
src/routes/login/+page.svelte	7-10, 50-54	<code>\$state()</code> y <code>bind:value</code>
src/lib/components/SubjectCard.svelte	5-19	<code>\$props()</code> con tipado
src/lib/stores/auth.js	1-24	Stores writable/derived
src/lib/stores/preloaded.ts	1-9	Stores precarga
src/routes/desarrollo-web-1/parcial-1/+page.svelte	10-40	Múltiples <code>\$state()</code>

Comandos

```
# Crear componente
touch src/lib/components/MiComponente.svelte

# Crear store
touch src/lib/stores/miStore.js
```

Teoría: Reactividad y estado en Svelte 5

Svelte 5 introdujo los **runes**, un nuevo sistema de reactividad explícito:

`$state()` — Declara una variable reactiva:

```
<script lang="ts">
  let count = $state(0);      // número reactivo
  let items = $state([]);     // array reactivo
  let user = $state(null);    // objeto reactivo
</script>
```

Las variables `$state` son profundamente reactivas. Al cambiar, el DOM se actualiza automáticamente.

`$derived()` — Valor calculado a partir de otros estados:

```
let doubled = $derived(count * 2);
```

Se recalcula automáticamente cuando cambian sus dependencias.

`$effect()` — Ejecuta código cuando cambian sus dependencias:

```
$effect(() => {
  console.log('count cambió a', count);
});
```

`$props()` — Recibe propiedades desde el componente padre (reemplaza `export let` de Svelte 4):

```
let { name, age = 0 }: { name: string; age?: number } = $props();
```

Stores (`writable`, `derived`) — Estado global accesible desde cualquier componente:

```
import { writable, derived } from 'svelte/store';
export const miStore = writable(valorInicial);
```

En el template se accede con prefijo `$`: `{ $miStore }`

Data Binding con `bind:` — Sincronización bidireccional:

- `bind:value={variable}` — Inputs de texto/número
- `bind:checked={variable}` — Checkboxes
- `bind:group={variable}` — Radio buttons
- `bind:files={variable}` — File inputs

3.1 `$state()` — `login/+page.svelte:7-10`

```
<script lang="ts">
  let username = $state('admin');
  let password = $state('');
  let error = $state('');
  let loading = $state(false);
  let grades = $state<any[]>([]);
  let showDetail = $state(false);
  let timeLeft = $state(45 * 60);
</script>
```

3.2 `$props()` — `SubjectCard.svelte:5-19`

```
<script lang="ts">
  let {
    icon, title, description,
    href = '/', boldTitle = false, noButton = false
  }: {
    icon: string; title: string; description: string;
    href?: string; boldTitle?: boolean; noButton?: boolean;
  } = $props();
</script>
```

3.3 `bind:value` — `login/+page.svelte:47-55`

```
<form onsubmit={handleSubmit}>
  <label>Usuario <input type="text" bind:value={username} /></label>
  <label>Contraseña <input type="password" bind:value={password} /></label>
  <button type="submit" disabled={loading}>
    {loading ? 'Ingresando...' : 'Ingresar'}
  </button>
</form>
```

3.4 Stores — `auth.js:1-24`

```
import { writable, derived } from 'svelte/store';

export const token = writable(storedToken);
export const currentUser = writable(null);
export const isAuthenticated = derived(currentUser, ($user) => $user !== null);
export const isAdmin = derived(currentUser, ($user) => $user?.role === 'admin');

export function login(userData, tokenValue) {
  localStorage.setItem('token', tokenValue);
  localStorage.setItem('user', JSON.stringify(userData));
  token.set(tokenValue);
  currentUser.set(userData);
}
```

Uso: `{#if $isAuthenticated} Bienvenido, {$currentUser?.full_name} {/if}`

3.5 `$derived` — parcial-1/+page.svelte:40,94

```
<script lang="ts">
  let isUnlimited = $derived($currentUser?.email === 'coordinacion@cinarsistemas.edu.co');
  let slots = $derived.by(() => getAvailableSlots());
</script>
```

Tema 4 — Interactividad (20 min)

Implementación

Archivo	Líneas	Rol
src/routes/+page.svelte	36-45, 58-62, 156	onclick , onkeydown
src/routes/login/+page.svelte	12-26, 47	onsubmit async
src/routes/Header.svelte	7-10, 27, 34	onclick botones
src/routes/desarrollo-web-1/+page.svelte	5-7	\$effect()
src/routes/desarrollo-web-1/parcial-1/+page.svelte	96-108, 160-171	\$effect() + timer
src/routes/desarrollo-web-1/enlaces-de-consulta/+page.svelte	61-66	onmousemove
src/lib/components/SubjectCard.svelte	28-71, 78	use:tilt , eventos

Teoría: Eventos y ciclo de vida en Svelte

Eventos del DOM se escuchan con directivas `on:` :

- `onclick={handler}` — Click del mouse
- `ondblclick={handler}` — Doble click
- `onmousemove={handler}` — Movimiento del mouse
- `onmouseenter` / `onmouseleave` — Entrar/salir del elemento
- `onkeydown={handler}` — Tecla presionada
- `onkeyup={handler}` — Tecla liberada
- `onsubmit={handler}` — Envío de formulario

- `oninput={handler}` — Cambio en input
- `onfocus` / `onblur` — Foco ganado/perdido
- `onscroll={handler}` — Scroll

Modificadores de eventos:

- `onclick|preventDefault={handler}` — Llama `e.preventDefault()`
- `onclick|stopPropagation={handler}` — Llama `e.stopPropagation()`
- `onclick|self={handler}` — Solo si el target es el elemento mismo
- Se pueden encadenar: `onclick|preventDefault|stopPropagation`

Ciclo de vida:

- `onMount(fn)` — Se ejecuta una vez al montar el componente
- `onDestroy(fn)` — Se ejecuta al destruir el componente (limpieza)
- `$effect(fn)` — Se ejecuta cuando cambian sus dependencias (Svelte 5)

Funciones en eventos:

```
<button onclick={() => showModal = true}>Abrir</button> <!-- inline -->
<button onclick={handleClick}>Click</button> <!-- por referencia -->
```

`$effect()` es el reemplazo moderno de `$:` (declaraciones reactivas de Svelte 4). Se ejecuta automáticamente cuando cualquiera de sus dependencias cambia.

4.1 `onclick` — +page.svelte:156, Header.svelte:34

```
<button onclick={() => showDetail = false}>Cerrar</button>
<button onclick={handleLogout}>Cerrar Sesión</button>
```

4.2 `onkeydown` — +page.svelte:36-45

```
<script lang="ts">
  function handleOverlayKeydown(e: KeyboardEvent) {
    if (e.key === 'Escape') showDetail = false;
  }
</script>
<div onkeydown={handleOverlayKeydown} tabindex="-1">
```

4.3 `onsubmit` — login/+page.svelte:12-26

```
<script lang="ts">
  async function handleSubmit(e: Event) {
    e.preventDefault();
    loading = true;
    try {
      const data = await authApi.login(username, password);
      authLogin(data.user, data.token);
      goto('/');
    } catch (err) {
      error = err instanceof Error ? err.message : 'Error';
    } finally { loading = false; }
  }
</script>
<form onsubmit={handleSubmit}>
```


4.4 `$effect()` — desarrollo-web-1/+page.svelte:5-7

```
<script lang="ts">
  // Redirecciona automáticamente si no está autenticado
  $effect(() => {
    if (!$isAuthenticated) goto('/login');
  });
</script>
```

```
<!-- parcial-1/+page.svelte:96-100 -->
// Autoenvío cuando el tiempo llega a 0
$effect(() => {
  if (timeLeft <= 0 && started && !finished) handleSubmit();
});
```

4.5 `onMount` — +page.svelte:19-34

```
<script lang="ts">
  import { onMount } from 'svelte';

  onMount(async () => {
    const result = await gradesApi.getMine();
    grades = result;
  });
</script>
```

Tema 5 — Estructuras de Control (20 min)

📌 Implementación

Archivo	Líneas	Rol
src/routes/+layout.svelte	19-35	<code>{#if}</code> indicadores
src/routes/+layout.svelte	34	<code>@render children()</code>
src/routes/+page.svelte	106-272	<code>{#if}</code> , <code>{#each}</code> anidados
src/routes/Header.svelte	19-40	<code>{#if}</code> auth/no-auth
src/routes/dashboard/+page.svelte	252-421	<code>{#if}</code> , <code>{#each}</code>
src/routes/desarrollo-web-1/enlaces-de-consulta/+page.svelte	85-87, 114-120	<code>{#each}</code> partículas

📖 Teoría: Bloques de control en Svelte

Svelte tiene su propia sintaxis de bloques de control, usando llaves `{}` :

`{#if}` — Renderizado condicional:

```

{#if condicion}
  <p>Se renderiza si es true</p>
{:else if otraCondicion}
  <p>Se renderiza si la primera es false y esta true</p>
{:else}
  <p>Se renderiza si todas son false</p>
{/if}

```

`{#each}` — Iteración sobre arrays:

```

{#each items as item}
  <p>{item.name}</p>
{/each}

{#each items as item, index}
  <p>{index}: {item.name}</p>
{/each}

{#each items as item (item.id)}      <!-- con key para eficiencia -->
  <p>{item.name}</p>
{/each}

{#each items as item, index (item.id)}
  <p>{index}: {item.name}</p>
{/each}

```

`{#await}` — Manejo de promesas:

```

{#await promesa}
  <p>Cargando...</p>
{:then resultado}
  <p>{resultado}</p>
{:catch error}
  <p>{error.message}</p>
{/await}

```

`{@html}` — Renderizado de HTML crudo:

```

<p>{@html contenidoHtml}</p>

```

`{@render children()}` — **Renderizado de slots (Svelte 5)**: Reemplaza a `<slot />` de Svelte 4. Se usa en layouts y componentes que envuelven contenido.

5.1 `{#if}` — [+page.svelte:114-151](#)

```

{#if loading}
  <p class="loading">Cargando...</p>
{:else if error}
  <p class="error">{error}</p>
{:else if grades.length === 0}
  <p class="empty">Sin calificaciones</p>
{:else}
  <table>
    {#each grades as grade}
      <tr>
        <td>{grade.subject}</td>
        <td>{grade.score}</td>
        <td>{grade.max_score}</td>
      </tr>
    {/each}
  </table>
{/if}

```

5.2 {#if} con auth — Header.svelte:19-40

```

{#if $isAuthenticated}
  <ul>
    {#if $currentUser?.role === 'admin'}
      <li><a href="/dashboard">Calificaciones</a></li>
    {/if}
  </ul>
  <button onclick={handleLogout}>Cerrar Sesión</button>
{:else}
  <ul>
    <li><a href="/login">Iniciar Sesión</a></li>
  </ul>
{/if}

```

5.3 {#each} con índice — +page.svelte:195-253

```

{#each detailData.questions as q, i}
  <div class="detail-question">
    <span>Pregunta {i + 1}</span>
    <p>{q.question}</p>
    {#if q.type === 'mc'}
      {#each q.options as opt, oi}
        <div class="{q.studentAnswer === oi ? 'selected' : ''}">
          <span>{opt}</span>
        </div>
      {/each}
    {/if}
  </div>
{/each}

```

5.4 {@render children()} — +layout.svelte:34

```

<main>
  {@render children()}
</main>

```

Tema 6 — Conexión con Datos Externos (20 min)

📌 Implementación

Archivo	Líneas	Rol
src/lib/api.js	1-46	Cliente HTTP con fetch + JWT
src/lib/loaders/preloader.ts	1-113	Precarga con Promise.all
src/lib/stores/preloaded.ts	1-9	Stores precarga
src/routes/+page.ts	1-24	Load function
src/routes/dashboard/+page.svelte	72-95	CRUD completo
src/routes/login/+page.svelte	12-26	Login API
src/routes/desarrollo-web-1/algorithmia/+page.svelte	46-83	submitPractice

📖 Teoría: Conexión con APIs externas en SvelteKit

Fetch API nativo — SvelteKit usa `fetch` del navegador (o `fetch` adaptado en SSR):

```
const res = await fetch('https://api.example.com/data');
const data = await res.json();
```

Autenticación JWT — Token almacenado en `localStorage` y enviado en headers:

```
Authorization: Bearer <token>
```

Load functions — Carga de datos server-side o client-side antes de renderizar:

- `+page.ts` → CSR (se ejecuta en el cliente)
- `+page.server.ts` → SSR (se ejecuta en el servidor, token seguro)
- Los datos se pasan al componente como `let { data } = $props()`

Precarga con `Promise.all` — Carga paralela de múltiples recursos:

```
const [usuarios, productos] = await Promise.all([
  fetch('/api/users').then(r => r.json()),
  fetch('/api/products').then(r => r.json())
]);
```

Manejo de errores — Patrón try/catch/finally en operaciones asíncronas:

```
try {
  loading = true;
  data = await api.getData();
} catch (err) {
  error = err.message;
} finally {
  loading = false;
}
```

Variables de entorno — Para URLs de API sensibles:

```
VITE_API_URL=https://mi-api.com // Expuesta al cliente (VITE_)
API_SECRET=... // Solo servidor (sin VITE_)
```

6.1 Módulo API — api.js:1-46

```
export const API_URL = import.meta.env.VITE_API_URL || 'https://testcinar26bknd.onrender.com/api';

export function getToken() {
  return localStorage.getItem('token');
}

export async function api(method, path, data) {
  const token = getToken();
  const options = {
    method,
    headers: {
      'Content-Type': 'application/json',
      ...(token ? { Authorization: `Bearer ${token}` } : {})
    }
  };
  if (data) options.body = JSON.stringify(data);

  const res = await fetch(`${API_URL}${path}`, options);
  const json = await res.json();
  if (!res.ok) throw new Error(json.error || 'Error');
  return json;
}

export const authApi = {
  login: (u, p) => api('POST', '/auth/login', { username: u, password: p }),
  profile: () => api('GET', '/auth/profile')
};

export const gradesApi = {
  getAll: (p) => api('GET', `/grades?${new URLSearchParams(p)}`),
  getMine: () => api('GET', '/grades/mine'),
  create: (d) => api('POST', '/grades', d),
  update: (id, d) => api('PUT', `/grades/${id}`, d),
  delete: (id) => api('DELETE', `/grades/${id}`)
};
```

6.2 Load function — +page.ts:1-24

```
import { browser } from '$app/environment';
import type { PageLoad } from './$types';
import { gradesApi } from '$lib/api';

export const load: PageLoad = async () => {
  if (browser) {
    const grades = await gradesApi.getMine();
    return { grades };
  }
  return { grades: [] };
};
```

6.3 Llamada en `onMount` — +page.svelte:19-34

```
onMount(async () => {
  if (!$isAuthenticated) { goto('/login'); return; }
  try {
    const result = await gradesApi.getMine();
    grades = result;
  } catch (err) {
    error = err instanceof Error ? err.message : 'Error';
  } finally { loading = false; }
});
```

6.4 CRUD — dashboard/+page.svelte:72-95

```
async function handleSubmit(e: Event) {
  e.preventDefault();
  try {
    if (editingId) await gradesApi.update(editingId, formData);
    else await gradesApi.create(formData);
    grades = await gradesApi.getAll({});
  } catch (err) {
    error = err instanceof Error ? err.message : 'Error';
  }
}

async function handleDelete(id: string) {
  if (!confirm('¿Eliminar?')) return;
  await gradesApi.delete(id);
  grades = await gradesApi.getAll({});
}
```

6.5 Preloader — preloader.ts:27-87

```
export async function wakeBackend(maxAttempts = 3) {
  for (let i = 0; i < maxAttempts; i++) {
    try {
      await fetch(BASE_URL, { signal: AbortSignal.timeout(20000) });
      break;
    } catch {
      await new Promise(r => setTimeout(r, 3000));
    }
  }
}

export async function preloadAll() {
  await wakeBackend();
  const [profile, myGrades] = await Promise.all([
    authApi.profile().catch(() => null),
    gradesApi.getMine().catch(() => null)
  ]);
}
```

6.6 `{#await}` (alternativa nativa)

```

{#await promise}
  <p>Cargando...</p>
{:then data}
  <p>Datos: {data}</p>
{:catch error}
  <p>Error: {error.message}</p>
{/await}

```

Resumen: Arquitectura del Proyecto

```

testcinar26/
├─ src/
│  ├─ app.html                # Tema 2
│  ├─ routes/
│  │  ├─ +layout.svelte      # Tema 1, 5
│  │  ├─ +layout.ts          # Tema 2, 6
│  │  ├─ +page.svelte        # Temas 1, 3, 4, 5, 6
│  │  ├─ +page.ts            # Tema 2, 6
│  │  ├─ Header.svelte       # Tema 1, 4, 5
│  │  ├─ login/+page.svelte  # Temas 1-6
│  │  ├─ dashboard/+page.svelte # Temas 1-6
│  │  └─ desarrollo-web-1/
│  │     ├─ +page.svelte      # Tema 1, 2, 4, 5
│  │     ├─ parcial-1/+page.svelte # Temas 1-6
│  │     ├─ parcial-1/+page.ts # Tema 2, 6
│  │     ├─ algoritmia/+page.svelte # Temas 2-6
│  │     └─ enlaces-de-consulta/ # Tema 2, 4, 5
│  └─ lib/
│     ├─ api.js               # Tema 6
│     ├─ exam.js              # Tema 6
│     ├─ components/SubjectCard.svelte # Tema 1, 3, 4, 5
│     ├─ stores/auth.js       # Tema 3
│     ├─ stores/preloaded.ts   # Tema 3, 6
│     ├─ loaders/preloader.ts  # Tema 6
│     └─ data/                # Datos estáticos
├─ static/                    # Archivos estáticos
├─ svelte.config.js
├─ vite.config.ts
└─ tsconfig.json

```

Mapa rápido

Tema	Archivos clave	Teoría cubierta
1. Navegación	<code>+layout.svelte</code> , <code>Header.svelte</code> , <code>goto()</code> , <code><a></code>	Filesystem routing, layouts, archivos <code>+</code> , rutas dinámicas
2. SEO	<code>app.html</code> , <code><svelte:head></code> , <code>+page.ts</code> , <code>+layout.ts</code>	Placeholders, metadatos, load functions CSR/SSR
3. Datos	<code>\$state()</code> , <code>\$props()</code> , <code>bind:value</code> , <code>stores</code> , <code>\$derived</code>	Runes, reactividad, stores writable/derived, data binding
4. Interactividad	<code>onclick</code> , <code>onkeydown</code> , <code>\$effect()</code> , <code>onMount</code>	Eventos DOM, modificadores, ciclo de vida, efectos
5. Estructuras	<code>{#if}</code> , <code>{#each}</code> , <code>{@render children() }</code>	Bloques if/each/await, slots, renderizado condicional
6. APIs	<code>api.js</code> (fetch+JWT), load functions, <code>Promise.all</code>	Fetch, JWT, CRUD, precarga, manejo de errores

Documento generado a partir del proyecto `testcinar26` — Cinar Sistemas 2026